

Triangle Renderer

Design & Code-Flow Reference

Dual-backend (OpenGL 4.6 / Vulkan 1.3) 3-D scene renderer with PBR materials, order-independent transparency, environment mapping, and a multi-pass selection-highlight pipeline.

1 Project Overview

The renderer is a real-time 3-D viewer written in C++17 that supports two graphics backends — OpenGL 4.6 (DSA) and Vulkan 1.3 — sharing a single input/state layer. Both backends produce identical visual output: 100 rounded cubes arranged on a Fibonacci sphere, an optional user-loaded mesh, a skybox, and a selection-outline effect.

Key capabilities:

- Nine BRDF shading models per instance: Blinn-Phong, Toon/Cel, Gooch, Rim, Oren-Nayar, Minnaert, Cook-Torrance PBR, Ward Anisotropic, Sub-surface scattering.
- Image-based lighting (IBL): diffuse irradiance + specular pre-filtered env + BRDF LUT.
- Order-independent transparency (OIT) using weighted, blended OIT (McGuire & Bavoil).
- 8× MSAA toggle (Vulkan only).
- Selection highlight with occlusion-aware outline: lit highlight pass, masking pass, dilation/outline pass, blend pass.
- Runtime mesh loading: glTF 2.0 and OBJ.
- HDR equirectangular environment map loading with on-the-fly IBL pre-computation.
- Hot renderer swap at runtime (GL ↔ Vulkan).

1.1 Directory layout

File / Folder	Purpose
main.cpp	Entry point; renderer loop; GL/Vulkan switch
config.h	Window size, instance count, display modes, material IDs
math_types.h	Vec3, Quat, Mat4, Arcball; all math inline
geometry.h	Vertex, InstanceData, Material; rounded-cube generator; Fibonacci-sphere layout
input.h / .cpp	InputHandler: mouse pick, arcball drag, SceneInput state bag
renderer.h / renderer.cpp	Vulkan backend (Renderer class)
renderer_gl.h / renderer_gl.cpp	OpenGL backend (RendererGL class)
mesh_loader.h / .cpp	glTF and OBJ parsers (returns Vertex/index vectors)
texture_util.h	HDR loading (tinyexr), LDR loading (stb_image), IBL baking
ui_helpers.h	ImGui widget helpers (color pickers, combo boxes, etc.)
shaders/	GLSL 4.50 source for Vulkan; GLSL 4.60 inline strings for GL

2 Architecture

2.1 Dual-backend design

Both backends share:

- **InputHandler / SceneInput** — the entire scene state (camera arcball, selection sets, material defaults, light params, texture paths, outline settings). The struct is copied wholesale on renderer swap so no state is lost.
- **Geometry data** — *Vertex* and *InstanceData* arrays built once in CPU memory and uploaded to GPU by whichever backend is active.
- **Math library** — *math_types.h* (Vec3, Quat, Mat4, Arcball).

The main loop in *main.cpp* is intentionally simple:

```
while (true) { input.state.switchRenderer = false; if (useGL) RendererGL{}.run(input); // blocks
until close or swap else Renderer{}.run(input); if (!input.state.switchRenderer) break; useGL =
!useGL; }
```

Each renderer constructs itself, runs its own main loop, then destructs completely before the other starts — no shared GPU objects cross the boundary.

2.2 Core data structures

Type	Size	Fields / Notes
Vertex	32 B	pos[3], nrm[3], uv[2] — interleaved, binding 0
InstanceData	116 B	transform (Mat4), color[4], ambient, diffuse, specular, shininess, materialType, p0-p3
Material	—	CPU mirror of per-instance material fields; default applied to all cubes at startup
LightParams	28 B	pos[3], color[3], intensity — world-space
SceneInput	~160 B	Camera arcballs, zoom, selection sets, display mode, outline params, light, paths, IBL intensity
SceneUBO (GL)	208 B	mvp, mv, selColor, lightPos, lightColor, shaderMode, hasAlbedo, hasEnvMap, iblIntensity, selMask[4]
PushConst (Vk)	208 B	Identical layout to SceneUBO; pushed per draw call

2.3 Selection bitmask

Up to 128 cubes can be selected simultaneously. A `uint32_t selMask[4]` (128 bits) is built each frame from `SceneInput.selectedCubes` (a `std::set<int>`) and uploaded to GPU. The vertex shader reads the bit for `gl_InstanceID` and emits `isSelected` as a float varying. This drives both the in-scene tint and the highlight pipeline.

3 Render Pipeline

Both backends execute the same logical passes in the same order each frame. The table below describes each pass; subsequent sections give per-pass detail.

Pass	Name	Output	Description
0	Skybox	mainFbo / swapchain	Full-screen tri; equirect → cubemap direction lookup; tone-mapped.
1	Opaque geometry	mainFbo (color+depth)	Cubes + mesh, $\alpha \geq 0.99$. Blinn-Phong / PBR / toon ... + IBL. Selection tint.
1a	MSAA resolve (GL only)	mainFbo	Blit 8× MSAA renderbuffer → 1× mainFbo before OIT.
2	OIT accumulation	oitFbo (accum+reveal)	Transparent fragments ($\alpha < 0.99$): weighted-blend OIT accumulate.
3	OIT composite	mainFbo	Blend accum/reveal textures onto mainFbo using McGuire formula.
4	Highlight	highlightFbo (RGBA+depth)	Re-render selected objects. Fragment uses selection color as albedo, applies full PBR.
4.5	Masking	maskedHighlightFbo (RGBA)	Per-pixel depth compare: if selected-object depth > main-scene depth, $\alpha *= \text{occl}$
5	Outline	outlineFbo (RGBA)	Full-screen dilation: 12 max-neighbor samples binarize mask, dilate, subtract inner mask.
6	Blend	mainFbo	Alpha-blend outline texture onto main color buffer.
7	ImGui	mainFbo / swapchain	Dear ImGui overlaid last; rendered directly to back-buffer.

3.1 Pass 0 — Skybox

A single full-screen triangle (gl_VertexID trick, no VBO) reconstructs the world-space view ray per fragment via an *inverse(proj × rotation)* matrix passed as a push constant / uniform. The equirectangular HDR texture is sampled using a spherical UV mapping, tone-mapped with Reinhard, and gamma-corrected. The skybox is drawn with depth writes off and depth test off so it sits behind all geometry.

3.2 Pass 1 — Opaque geometry

The main vertex shader transforms each instance using its InstanceData.transform column-major matrix, outputs view-space position and normal, and passes per-instance material fields as varyings. The fragment shader routes to one of nine shadeSolid() branches based on fragMatType, then optionally calls applyIBL() for irradiance + specular IBL. Only fragments with $\alpha \geq 0.99$ are written; semi-transparent fragments are discarded and handled in Pass 2.

Display mode is selected via shaderMode:

shaderMode	Effect
0 (SOLID)	Full BRDF lighting + IBL
1 (WIREFRAME)	Output black; polygon offset used for solid+wire overlay
2 (UNLIT)	Output albedo directly, no lighting
3 (NORMALS)	Visualise view-space normal as RGB

3.3 Passes 2–3 — Order-independent transparency (OIT)

Weighted, blended OIT (McGuire & Bavoil 2013) is used when any instance has alpha < 0.99. Two render targets — **accum** (RGBA16F, additive blend) and **reveal** (R8, multiplicative blend) — are written in Pass 2 with depth writes off. Pass 3 composites them onto the opaque buffer using the formula:

```
vec3 avgColor = accum.rgb / max(accum.a, 1e-5); outColor = vec4(avgColor, 1.0 - reveal);
```

3.4 Pass 4 — Highlight

Selected cubes/mesh are re-rendered into a dedicated RGBA8 framebuffer with its own depth buffer. Unlike the main pass, **albedo is the selection color** (not the instance color), but the full per-instance `shadeSolid()` + `applyIBL()` lighting pipeline is applied so the highlight is physically shaded — it is not a flat overlay. Non-selected instances are discarded via the `selMask` bitmask check.

3.5 Pass 4.5 — Masking

A full-screen quad reads three textures: main-scene depth, highlight depth, and the raw highlight RGBA. For each pixel:

```
float a = src.a * ((highlightDepth > mainDepth + 0.0001) ? occlusionAlpha : 1.0); outMask =  
vec4(src.rgb, a);
```

This dims the highlight where the selected object is behind other geometry, producing the X-ray/ghost effect. The `occlusionAlpha` is user-adjustable (0–1).

3.6 Pass 5 — Outline

A dilation-based outline is computed from the masked highlight texture. The mask is first binarized (`step(0.001, alpha)`) so both visible and occluded selection regions are treated as inside. Twelve neighbour offsets at distance `thickness × 0.5 texels` are max-sampled:

```
#define S(off) step(0.001, texture(mask, uv + off * texelSize * s).a) float d = max(mask,  
max(S(vec2(1,0)), max(S(vec2(-1,0)), ... ))); float isOutline = clamp(d - mask, 0.0, 1.0);
```

The subtraction of the inner mask isolates the border ring. When *antialiased* mode is on, `smoothstep(0,1,isOutline)` feeds `mix()` for a sub-pixel soft edge; otherwise a hard >0.5 threshold is used.

3.7 Pass 6 — Blend

The outline texture is alpha-blended onto *mainFbo* using standard `src-alpha / one-minus-src-alpha` blending. No depth test is performed.

4 Shading Models

All shading is implemented in the shared `shadeSolid()` function inside the fragment shader. The function signature is identical in all relevant shaders (opaque, OIT, highlight):

```
vec3 shadeSolid(vec3 N, vec3 L, vec3 viewDir, vec3 baseColor, float ambient, float diffuse, float specular, float shininess, int matType, vec4 p)
```

MAT ID	Name	p0	p1	p2	p3
0	Blinn-Phong	—	—	—	—
1	Toon / Cel	bands	spec thresh	rim power	rim strength
2	Gooch	color mix	spec scale	—	—
3	Rim	rim power	rim R	rim G	rim B
4	Oren-Nayar	σ (roughness)	—	—	—
5	Minnaert	k (retro)	—	—	—
6	Cook-Torrance PBR	metalness	roughness	—	—
7	Ward Anisotropic	α_x	α_y	rot angle°	spec scale
8	Sub-surface	wrap	back-lit	scatter	—

4.1 IBL pipeline

Image-based lighting is added by `applyIBL()` after `shadeSolid()`:

- Diffuse IBL: irradiance cubemap sampled along the world-space normal (reconstructed via transpose of the upper-left 3×3 of the view matrix).
- Specular IBL (PBR only, `matType==6`): pre-filtered environment cubemap sampled at `roughness × 4 LOD + BRDF integration LUT lookup`.
- Both contributions are scaled by `iblIntensity`.

The IBL textures are pre-computed from the loaded HDR equirectangular map at load time using a CPU-side baking step (`texture_util.h`).

5 OpenGL Backend (RendererGL)

Uses OpenGL 4.6 with Direct State Access (DSA). All shader source is embedded as raw-string literals in *renderer_gl.cpp*. GLSL 4.60 core.

5.1 Programs

Program	Vertex shader	Fragment shader
progMain	MAIN_VERT	SHADE_FUNC + OPAQUE_FRAG_MAIN
progOit	MAIN_VERT	SHADE_FUNC + OIT_FRAG_MAIN
progComposite	COMPOSITE_VERT	COMPOSITE_FRAG
progSkybox	SKYBOX_VERT	SKYBOX_FRAG
progHighlight	MAIN_VERT	SHADE_FUNC + HIGHLIGHT_FRAG_MAIN
progMasking	MASKING_VERT (quad)	MASKING_FRAG
progOutline	OUTLINE_VERT (quad)	OUTLINE_FRAG
progBlend	MASKING_VERT (quad)	BLEND_FRAG

SHADE_FUNC is a shared GLSL string declaring the SceneUBO, all texture samplers, all fragment-shader inputs (locations 0–8), and both `shadeSolid()` and `applyIBL()`. It is prepended to each fragment main before compilation, avoiding duplication.

5.2 Framebuffer objects

FBO	Attachments	Consumer
mainFbo	mainColorTex + depthTex	Blitted to default FB
msaaFbo	msaaColorRbo + msaaDepthRbo (×8)	Resolved → mainFbo
oitFbo	accumTex + revealTex (shares depthTex)	progComposite
highlightFbo	highlightTex + highlightDepthTex	progMasking
maskingFbo	maskedHighlightTex	progOutline
outlineFbo	outlineTex	progBlend

5.3 State transitions per frame

Each pass explicitly sets its required depth/blend/mask state before drawing. Key transitions:

- Pass 1 (opaque): depth test ON, depth write ON, blend OFF.
- Pass 2 (OIT accum): depth write OFF, blend ON with separate blend funcs (ONE/ONE for accum; ZERO/ONE_MINUS_SRC_COLOR for reveal).
- Pass 3 (OIT composite): depth test OFF, standard alpha blend.

- Pass 4 (highlight): depth test ON, depth write ON (fresh buffer), blend OFF.
- Passes 4.5–5 (masking, outline): depth OFF, depth write OFF, blend OFF.
- Pass 6 (blend): alpha blend ON, depth OFF.

6 Vulkan Backend (Renderer)

Uses Vulkan 1.3 with validation layers in Debug builds. Shader SPIR-V is compiled from GLSL 4.50 by glslc at CMake build time and loaded from shaders/*.spv at runtime.

6.1 Initialization order

The `initVulkan()` method creates resources in strict dependency order:

```
createOpaqueRenderPass(); createGraphicsPipeline(); // pipelineLayout created here
createOITPipelines(); createHighlightPipeline(); createOutlinePipeline();
createMaskingPipeline(); createBlendPipeline(); createOITImages(); createOITDescriptorSets();
createOpaqueFramebuffers(); createOITFramebuffers(); ... createHighlightImages();
createHighlightFramebuffers(); createMaskingFramebuffers(); createMaskingDescriptorSets();
createOutlineDescriptorSets(); createBlendFramebuffers(); createBlendDescriptorSets();
createVertexBuffer(); createIndexBuffer(); createInstanceBuffer(); createCommandBuffers();
createSyncObjects(); initImGui();
```

pipelineLayout is created inside `createGraphicsPipeline()` and must exist before any pipeline that references it.

6.2 Pipeline layouts

Layout	Set 0	Push constants
pipelineLayout (main, highlight, styleDesc)	1 texture sampler: albedo, irradiance, prefilteredEnvironmentMap, emissiveColor, highlightColor	20 B: filter, equivSel, color, highlightColor, shaderMode, hasStyleDesc
outlinePipelineLayout	1 combined image sampler (masked highlight)	32 B: texelSize(xy), thickness, antialiased, outlineColor(rgba)
maskingPipelineLayout	3 samplers: mainDepth, highlightDepth, highlightMask	4 B: occlusionAlpha
blendPipelineLayout	1 combined image sampler (outline texture)	8 B: texelSize(xy)
compositePipelineLayout	2 samplers: accum, reveal	None

6.3 Swapchain recreation

The swapchain is torn down and rebuilt on window resize or `VK_ERROR_OUT_OF_DATE_KHR`. Per-swapchain resources (framebuffers, OIT images, highlight/masking/outline images, descriptor sets) are recreated. Persistent resources (render passes, pipelines, pipeline layouts, texture descriptors, sync objects) survive the rebuild.

6.4 MSAA (optional 8×)

When enabled, a depth prepass renders only depth to allow early-z rejection, then the opaque pass uses MSAA color and depth attachments with resolve to the swapchain image as part of the render pass. The OIT pass always uses 1× samples (shares the resolved depth).

7 Shader Reference

Source File	SPV name	Stage	Purpose
triangle.vert	vert.spv	Vert	Main geometry: transform, normals, instance material passthrough
triangle.frag	frag.spv	Frag	Opaque shading: shadeSolid + applyIBL; selection tint
oitaccum.frag	oitaccum.spv	Frag	OIT: same shading as triangle.frag; outputs accum/reveal; weight function
composite.vert	composite_vert.spv	Vert	Full-screen triangle (gl_VertexID)
composite.frag	composite_frag.spv	Frag	McGuire OIT composite: avgColor / max(a,eps), alpha = 1-reveal
skybox.vert	skybox_vert.spv	Vert	Full-screen tri; reconstructs world-space ray from inverse(proj×rot)
skybox.frag	—	Frag	Equirect lookup; Reinhard tonemap; gamma correct
highlight.vert	highlight_vert.spv	Vert	Same layout as triangle.vert; outputs all lighting varyings
highlight.frag	highlight_frag.spv	Frag	Lit highlight: albedo = selectionColor; full shadeSolid + applyIBL
masking.vert	masking_vert.spv	Vert	Passthrough quad (gl_VertexID)
masking.frag	masking_frag.spv	Frag	Depth compare; alpha *= occlusionAlpha if occluded
outline.vert	outline_vert.spv	Vert	Full-screen quad (gl_VertexID)
outline.frag	outline_frag.spv	Frag	Binarize + 12-sample max-dilation – inner mask; AA smoothstep or hard
blend.frag	—	Frag	Sample outline texture; pass through (blended by pipeline state)

7.1 Vertex attribute layout

Binding 0 — per-vertex (stride = 32 B):

Location	Name	Format	Offset
0	inPosition	RGB32F	0
9	inNormal	RGB32F	12
10	inUV	RG32F	24

Binding 1 — per-instance (stride = 116 B, divisor = 1):

Location	Name	Format	Offset (B)
1–4	instCol0–3 (transform)	4×RGBA32F	0
5	instColor	RGBA32F	64
6	instMaterial	RGBA32F	80 (ambient, diffuse, specular, shininess)
7	instMatType	R32_SINT	96
8	instMatParams	RGBA32F	100 (p0–p3)

8 Input & Camera

8.1 Mouse controls

Gesture	Effect
MMB drag	Arcball camera rotate
Shift + MMB drag	Camera pan (translate camTarget)
Ctrl + MMB drag	Zoom (change zoomDist)
Alt + MMB drag	Environment map rotate
Alt + Shift + MMB	Environment pan (second arcball)
Alt + Ctrl + MMB	Environment zoom (scale focal length)
Left click (cube)	Toggle cube selection
Left click (mesh)	Toggle mesh selection
Left click (empty)	Clear selection

8.2 Arcball

Both the camera and the environment use the same Arcball struct (*math_types.h*). On MMB press, the click is projected to a virtual unit sphere. During drag, the rotation quaternion is updated as the cross-product axis between the start and end sphere vectors. The resulting quaternion is converted to a column-major 4×4 matrix each frame.

8.3 Picking

Cube selection uses CPU ray–sphere intersection: the mouse position is unprojected to a view-space ray (using the inverse of `proj×view`), then tested against spheres of radius 0.6 centred at each cube's world position. Mesh picking tests against a bounding sphere computed at load time. Ctrl+click toggles multi-selection; plain click selects one and deselects others.

9 Code Summary

9.1 File-by-file

main.cpp (40 lines)

Entry point. Parses --opengl / --gl flag (default: OpenGL). Runs the renderer-swap loop. All exception handling terminates here.

config.h (34 lines)

Compile-time constants: window size (1500×1500), MAX_FRAMES_IN_FLIGHT (2), INSTANCE_COUNT (100), SPHERE_RADIUS (5.0). Declares DisplayMode enum and the nine MAT_* integer constants.

math_types.h (117 lines)

Header-only math library. Provides Vec3, Quat, Mat4 (column-major float[16]) with cross/dot/normalize, quaternion multiply, quatToMat4, mat4mul, lookAt, perspective (Vulkan convention: Y-flipped, depth [0,1]), mat4inverse, and the Arcball struct with sphere projection and drag update.

geometry.h (202 lines)

Declares Vertex (32 B), InstanceData (116 B), and Material. Provides:

- **generateRoundedCube()** — Procedural rounded-cube tessellator. 6 faces × N×N quads; edge-biased parameter remapping concentrates vertices at the corners; per-vertex normals point from the inner box surface.
- **buildInstanceData()** — Places INSTANCE_COUNT cubes on a Fibonacci sphere (golden-angle spiral), copies the supplied Material into each record.
- **hsvToRgb()** — Used historically; retained for colour generation.

input.h / input.cpp (~74 / ~120 lines)

InputHandler owns a SceneInput and handles GLFW callbacks. onMouseButton begins/ends arcball drags and dispatches to pickCube() / pickMesh(). onCursorPos updates all three arcballs depending on modifier key state. Pick context (viewport size and cube positions) is injected by the renderer each frame.

renderer_gl.cpp / renderer_gl.h (~1 400 / 152 lines)

Full OpenGL 4.6 renderer. Highlights:

- All GLSL source is embedded as C++ raw-string literals; SHADE_FUNC is prepended to opaque, OIT, and highlight fragment shaders to share the lighting code without duplication.
- createFbos(w, h) — Allocates all textures and FBOs; called at startup and on GLFW framebuffer resize.
- drawScene(shaderMode, oitPass) — Binds IBL textures, uploads UBO, draws cubes and (if loaded) mesh with the correct program.
- drawFrame() — Orchestrates all eight passes in order; each pass is wrapped in a glDebugMessageInsert marker for GPU profiler capture.
- renderUI() — ~300-line Dear ImGui panel: display mode, material sliders, light editor, outline controls, texture loaders, renderer switch.
- loadEnvMap() — Loads HDR via tinyexr/stb, bakes irradiance and pre-filtered cubemaps on CPU, uploads all five textures.

renderer.cpp / renderer.h (~3 700 / 315 lines)

Full Vulkan 1.3 renderer. Mirrors the GL feature set. Key differences from GL:

- Resource creation is explicit: each render pass, pipeline, descriptor set layout, descriptor pool, and framebuffer is created with a dedicated method.
- Instance buffer is persistently mapped (vkMapMemory) and updated every frame with a memcpy.
- recordCommandBuffer() — ~500 lines encoding all eight passes, barriers between passes, and ImGui draw data.
- Swapchain recreation on resize: cleanupSwapchain() destroys extent-dependent resources; recreateSwapchain() rebuilds them.
- Validation layers enabled in Debug builds; VK_EXT_debug_utils used for per-pass GPU labels.

mesh_loader.cpp (~250 lines)

Parses glTF 2.0 (binary and ASCII) and OBJ files into the shared Vertex / uint32_t index vector format. Normals are computed from cross-products if absent. Uses a minimal inline glTF parser (no external library dependency).

texture_util.h (~400 lines)

Header-only texture utilities:

- HDR loading via tinyexr (EXR) and stb_image (HDR).
- LDR loading via stb_image (PNG/JPG).
- CPU-side IBL baking: spherical-harmonic irradiance cubemap, GGX split-sum pre-filtered env cubemap (5 mip levels), BRDF integration LUT.
- Equirectangular → cubemap direction sampling.

9.2 Build system (CMakeLists.txt)

CMake 3.20+. Single target triangle. Key steps:

- Finds Vulkan SDK (including glslc).
- Compiles every GLSL shader in shaders/ to SPIR-V via add_custom_command; output goes to build/shaders/.
- Copies SPV files to \$<TARGET_FILE_DIR>/shaders/ so the executable can find them via relative path shaders/*.spv.
- Links: Vulkan, GLFW, glad (OpenGL loader), Dear ImGui (source files included).
- Compile flags: C++17, /W3 (MSVC).

9.3 Dependency graph

Module	Depends on
main.cpp	input.h, renderer.h, renderer_gl.h
renderer.cpp	input.h, geometry.h, math_types.h, config.h, mesh_loader.h, texture_util.h, ui_helpers.h, ImGui, Vulkan, GLFW
renderer_gl.cpp	input.h, geometry.h, math_types.h, config.h, mesh_loader.h, texture_util.h, ui_helpers.h, ImGui, glad, GLFW

Module	Depends on
mesh_loader.cpp	geometry.h
input.cpp	input.h, math_types.h, geometry.h, config.h
geometry.h	config.h, math_types.h
texture_util.h	stb_image (header-only), tinyexr (header-only)

10 Glossary

Term	Definition
Arcball	Virtual trackball: maps 2-D mouse motion to 3-D rotation via sphere-surface projection and quaternion composition.
DSA	Direct State Access — OpenGL 4.5+ API that operates on named objects directly without binding to a global target first.
FBO	Framebuffer Object — off-screen render target in OpenGL.
Fibonacci sphere	Uniform distribution of N points on a sphere via the golden-angle spiral; produces even spacing with no poles.
IBL	Image-Based Lighting — pre-baked irradiance / specular environment maps used for ambient and specular contributions.
OIT	Order-Independent Transparency — technique to composite transparent fragments without sorting.
occlusionAlpha	User-controllable alpha (0–1) applied to selected-object regions that are occluded by the main scene; produces the X-ray effect.
pipelineLayout	Vulkan object describing push-constant ranges and descriptor set layouts used by a pipeline.
push constant	Fast per-draw GPU data path in Vulkan — up to 128 B updated directly in the command buffer.
SPIR-V	Binary intermediate shader language consumed by Vulkan; produced from GLSL by glslc.
texDescSet	Vulkan descriptor set (set=0) binding all five scene textures (albedo, irradiance, prefilteredEnv, brdfLUT, equirect).
UBO	Uniform Buffer Object — per-frame data block uploaded via a mapped buffer; equivalent to Vulkan push constants in OpenGL.
Weighted OIT	McGuire & Bavoil 2013: each fragment contributes to an accum RGBA and a reveal R buffer; composited analytically.